

Assignment: DDL, DML & DCL — Hands-on pgAdmin Practice

Note: Use the college_db database and the students table you created on previous assignment. If you did not complete previous, create college_db with an academics schema and a students table before starting. Write and save every SQL statement you run in pgAdmin's Query Tool.

Hands-on pgAdmin Practice

Q1. Add Columns (DDL) — Add two new columns to the students table: phone_number (VARCHAR(15)) and department (VARCHAR(50)).

```
-- Write your ALTER TABLE statement(s) here:  
ALTER TABLE STUDENT ADD phone_number VARCHAR(15) , department VARCHAR(50);
```

Q2. Rename a Column (DDL) — Rename the column full_name to student_name in the students table.

```
-- Write your ALTER TABLE statement here:  
ALTER TABLE STUDENT RENAME COLUMN full_name TO student_name;
```

Q3. Change a Column's Data Type (DDL) — Change the data type of cgpa to NUMERIC(4,2) so it can store values like 9.50.

```
-- Write your ALTER TABLE statement here:  
ALTER TABLE STUDENT ALTER COLUMN cgpa TYPE NUMERIC(4,2);
```

Q4. Create a New Table (DDL) — Inside the academics schema, create a new table named courses with columns: course_id (SERIAL PRIMARY KEY), course_name (VARCHAR(100)), credits (SMALLINT), and department (VARCHAR(50)).

```
-- Write your CREATE TABLE statement here:  
CREATE TABLE academics.courses  
(  
  course_id SERIAL PRIMARY KEY,  
  course_name VARCHAR(100),  
  credits SMALLINT ,  
  department VARCHAR(50)  
);
```

Q5. Insert Multiple Rows (DML) — Insert at least 5 rows into the students table with realistic values for every column, using a single multi-row INSERT statement.

```
-- Write your INSERT statement here:  
INSERT INTO students  
  (student_id, student_name, email, phone_number, department, cgpa)  
VALUES
```

```
(1, 'Alice Johnson', 'alice@gmail.com', '9876543210', 'Computer Science', 9.50),
(2, 'Bob Smith', 'bob@gmail.com', '9876543211', 'Mechanical', 8.75),
(3, 'Charlie Brown', 'charlie@gmail.com', '9876543212', 'Electrical', 8.90),
(4, 'David Wilson', 'david@gmail.com', '9876543213', 'Civil', 7.85),
(5, 'Emma Davis', 'emma@gmail.com', '9876543214', 'Computer Science', 9.20);
```

Q6. Insert Data into courses (DML) — Insert at least 4 rows into the courses table you created in Q4.

```
-- Write your INSERT statement here:
INSERT INTO courses (course_name, credits, department)
VALUES
    ('Mathematics', 5, 'CSE'),
    ('Data Structures', 4, 'CSE'),
    ('Database Systems', 4, 'CSE'),
    ('Computer Networks', 3, 'CSE');
```

Q7. Update Records (DML) — Update the department to 'Computer Science' for all students where cgpa is greater than 8.0.

```
-- Write your UPDATE statement here:
UPDATE students
SET department = 'Computer Science'
WHERE cgpa > 8.0;
```

Q8. Delete Records (DML) — Delete any student row where is_active is FALSE.

```
-- Write your DELETE statement here:
DELETE from students WHERE is_active = FALSE;
```

Q9. SELECT Queries (DML/DQL) — Write two queries: (a) list all students sorted by cgpa in descending order, and (b) list only student_name and email for students in the 'Computer Science' department.

```
-- Query (a):
SELECT * FROM students ORDERBY cgpa DESC;
-- Query (b):
SELECT student_name , email FROM students WHERE department = 'Computer Science';
```

Q10. Create a Role & Grant Privileges (DCL) — Create a new role named readonly_user with a login password of your choice. Grant it SELECT privilege on the students table and USAGE on the academics schema.

```
-- Write your CREATE ROLE and GRANT statements here:
CREATE ROLE readonly_user
WITH LOGIN
PASSWORD 'MyPassword123';

GRANT USAGE ON SCHEMA academics TO readonly_user;

GRANT SELECT ON TABLE academics.students TO readonly_user;
```

Q11. Revoke Privileges (DCL) — Revoke the SELECT privilege from readonly_user on the students table, then confirm (by describing what would happen) if readonly_user tries to run a SELECT query afterward.

```
-- Write your REVOKE statement here:
REVOKE SELECT ON TABLE academics.students
FROM readonly_user;

//if user tries to make SELECT operation
-> ERROR: permission denied for table students
```

Q12. TRUNCATE Practice (DDL) — Create a temporary copy table named students_copy (same structure as students, no data). Insert 2-3 test rows into it, then run TRUNCATE TABLE students_copy RESTART IDENTITY and confirm the table is empty with a SELECT.

```
-- Write your CREATE TABLE, INSERT, TRUNCATE and SELECT statements here:
// Create a copy of students with the same structure
CREATE TABLE students_copy (LIKE students);

// Insert test rows
INSERT INTO students_copy (student_name, email, phone_number, department, cgpa,
is_active)
VALUES
    ('Test Student 1', 'test1@gmail.com', '9876543210', 'Computer Science', 8.50,
TRUE),
    ('Test Student 2', 'test2@gmail.com', '9876543211', 'Mechanical', 7.80, TRUE),
    ('Test Student 3', 'test3@gmail.com', '9876543212', 'Civil', 8.20, FALSE);

// Remove all rows and restart identity
TRUNCATE TABLE students_copy RESTART IDENTITY;

//Confirm table is empty
SELECT *
FROM students_copy;
```